Git-workflow для учебного проекта «Система заявок»

1. Рекомендуемый workflow (упрощённый GitHub Flow)

Для лабораторной работы оптимален **упрощённый GitHub Flow** с элементами feature branching:

- `main` — стабильная ветка, всегда «работает» (защищена от прямых пушей в реальных проектах)
- `develop` — ветка интеграции (опционально; для лабы можно обойтись без неё)
- `feature/<название>` — по одной ветке на функциональность
- `fix/<название>` — исправления
- Слияние — через `--no-ff` (сохраняет историю ветвления) или PR в GitHub

Правила для лабы:

1. Один коммит = одно логическое изменение (атомарность)
2. Сообщение коммита: `тип(область): краткое описание` (Conventional Commits)
3. Перед слиянием — `git pull --rebase` или merge `main` в ветку
4. Ветки удаляются после слияния

---

2. Содержательные сообщения коммитов (Conventional Commits)

Формат: `<тип>(<область>): <описание в императиве>`

Тип	Назначение	Пример
`feat`	новая функциональность	`feat(tickets): add ticket creation form`
`fix`	исправление бага	`fix(auth): correct password validation regex`
`docs`	документация	`docs(readme): add setup instructions`
`test`	тесты	`test(tickets): cover status transition rules`
`refactor`	рефакторинг без изменения поведения	`refactor(api): extract ticket service layer`
`chore`	служебное (зависимости, конфиги)	`chore(deps): add django-rest-framework`

Тип	Назначение	Пример
`style`	форматирование	`style: apply black formatting`
`ci`	CI/CD	`ci: add pytest workflow`

Примеры хороших сообщений по этапам проекта:

text

```
chore(init): initialize project structure and dependencies
docs(readme): describe system architecture and request lifecycle
feat(models): add Ticket, User and Comment entities
feat(tickets): implement ticket creation endpoint POST /tickets
feat(tickets): add status transition validation (new->in_progress->closed)
feat(ui): add ticket list page with filters by status
fix(tickets): prevent closing ticket without assignee
test(tickets): add unit tests for status transition rules
docs(api): document REST endpoints in openapi.yaml
```

---

3. `.gitignore`

gitignore

```
# Python
__pycache__/
*.py[cod]
*.egg-info/
.venv/
venv/
env/
.pytest_cache/
.coverage
htmlcov/

# Node (если фронтенд)
node_modules/
dist/
build/
*.log

# IDE / OS
.idea/
.vscode/
*.swp
.DS_Store
Thumbs.db

# Секреты и локальные конфиги
.env
.env.*
!.env.example
*.sqlite3
db.sqlite3

# Отчёты по лабе (если генерируются)
*.pdf
!docs/report.pdf
```

---

4. Последовательность git-команд: создание репозитория и коммиты

bash

```
# 1. Инициализация
mkdir ticket-system && cd ticket-system
git init -b main
git config user.name  "Иван Иванов"
git config user.email "ivan@example.com"

# 2. Первый коммит: структура + .gitignore
# (создать файлы README.md, .gitignore, requirements.txt)
git add .gitignore README.md requirements.txt
git commit -m "chore(init): initialize project structure and dependencies"

# 3. Ещё несколько коммитов в main (базовая заготовка)
git add docs/architecture.md
git commit -m "docs(readme): describe system architecture and request lifecycle"

git add app/models.py
git commit -m "feat(models): add Ticket, User and Comment entities"

# 4. Создание удалённого репозитория (GitHub CLI)
gh repo create ticket-system --public --source=. --push
# или вручную:
# git remote add origin git@github.com:user/ticket-system.git
# git push -u origin main
```

---

5. Работа с ветками и слияние

bash

```
# --- Ветка feature: создание заявки ---
git switch -c feature/ticket-creation
# ... правки ...
git add app/api/tickets.py app/serializers.py
git commit -m "feat(tickets): implement ticket creation endpoint POST /tickets"

git add tests/test_tickets.py
git commit -m "test(tickets): add unit tests for ticket creation"

# Перед слиянием — обновить ветку от main
git switch main
git pull origin main
git switch feature/ticket-creation
git rebase main            # или: git merge main

# Слияние в main с сохранением истории ветки
git switch main
git merge --no-ff feature/ticket-creation -m "merge: feature/ticket-creation
into main"

# Пуш и удаление ветки
git push origin main
git branch -d feature/ticket-creation
git push origin --delete feature/ticket-creation

# --- Вторая ветка: список заявок ---
```

```
git switch -c feature/ticket-list
# ... правки ...
git add app/api/tickets.py
git commit -m "feat(tickets): add ticket list endpoint with status filter"
git switch main
git merge --no-ff feature/ticket-list -m "merge: feature/ticket-list into main"
```

---

6. Моделирование конфликта при слиянии и его разрешение

Сценарий: две ветки правят одну и ту же строку в `app/api/tickets.py`.

bash

```
# --- Ветка A ---
git switch -c feature/status-validation
# правим строку STATUSES = [...] в app/api/tickets.py
git add app/api/tickets.py
git commit -m "feat(tickets): add 'rejected' status to allowed transitions"

# --- Ветка B от main (до слияния A) ---
git switch main
git switch -c feature/priority-field
# правим ТУ ЖЕ строку STATUSES = [...] иначе
git add app/api/tickets.py
git commit -m "feat(tickets): add 'on_hold' status for pending customer reply"

# --- Слияние A в main ---
git switch main
git merge --no-ff feature/status-validation -m "merge: feature/status-
validation"

# --- Слияние B — возникает конфликт ---
git switch main
git merge --no-ff feature/priority-field
# → CONFLICT (content): Merge conflict in app/api/tickets.py
#   Automatic merge failed; fix conflicts and then commit the result.

# --- Разрешение ---
git status                       # список конфликтных файлов
git diff                         # посмотреть маркеры <<<<<<< ======= >>>>>>>
# правим файл вручную: оставляем ОБА изменения (rejected + on_hold)
# удаляем маркеры <<<<<<<, =======, >>>>>>> и лишнее

git add app/api/tickets.py       # помечаем как разрешённый
git commit -m "merge: resolve conflict between status-validation and priority-
field"

# --- Альтернатива: прервать слияние ---
# git merge --abort
```

Проверка после разрешения:

bash

```
git log --oneline --graph --all   # увидеть структуру слияний
git diff HEAD~1                   # убедиться, что изменения корректны
```

---

7. Полезные команды для отчёта

bash

```
git log --oneline --graph --decorate --all   # граф истории для отчёта
git log --pretty=format:"%h %ad %s" --date=short
git shortlog -sn                              # вклад участников
git branch -a                                 # все ветки
git reflog                                    # история перемещений HEAD
```

Итоговая схема для отчёта:

